

UI task 1

Mogula Maksim

October 2026

Contents

1	Introduction	3
2	User guide and project structure explained	3
2.1	Project files	3
3	PEAS description	4
4	Environment classification	5
5	Representation of the environment, percepts and actions	5
5.1	Environment	5
5.2	Actuator	5
5.3	Sensor and percept	6
5.4	Episode loop	6
5.5	Visual representation	6
6	Performance measure	7
7	Baseline agent	7
8	Memory agent	7
8.1	Decision rule	7
8.2	Example	7
8.2.1	What happened	8
8.2.2	Cost of the roads at the moment of each decision	8
8.2.3	Whole episode	9
8.3	Internal state	9
9	Rationality, autonomy and limitations	10

10 Experiments	11
10.1 Setup	11
10.2 Configurations	11
10.3 Results	12
10.4 Interpretation	13
11 Weaknesses and possible improvements	14
12 Ending and main point of project	14
13 Libraries and external sources	15

1 Introduction

The goal of this project is to design, implement and compare an agent without memory to the one that has. The chosen theme is a **courier**: the courier starts at the base, has to deliver a package to every city on the map and return back to the base, while making as little steps as possible and also trying to have the shortest route.

All cities are connected with each other, but some roads are bad. A trip on a bad road can fail – the courier does not get through, stays in the same city and the cost of the trip is still paid. The agent does not know which roads are bad, it can only notice that a trip failed. The chance to fail on a bad road is written in config.py.

Two agents are compared:

- **BaselineAgent** – always goes to the nearest unvisited city, has no memory, if it fails on a road, it will try it until it gets to the next city
- **MemoryAgent** – uses the same rule(algorithm), but remembers failed trips and thinks those roads as more expensive. Calculates this using easy formula: $\text{road} / (1 - \text{chance of failure})$

Both agents use the same decision rule, the only difference is the memory. Actually firstly i had another algorithm in memoryAgent and it was 2-opt algorithm, but then i decided that it would be unfair, because my addition of memory will show nothing, because 2-opt algorithm will be better at any time. This makes it possible to measure exactly how much the memory helps.

2 User guide and project structure explained

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

```
python main.py          # one run, draws the routes
python experiment.py     # both agents on many seeds
```

2.1 Project files

city.py contains the **City** dataclass. A city has an index, x and y coordinates and a name. The class is frozen, so a city cannot be changed after the map is created.

percept.py contains the **Percept** dataclass, which is everything the agent is allowed to see in one step: the current city, the list of cities, the set of visited cities, whether the last trip succeeded and the distance matrix. It is frozen and the visited cities are a **frozenset**, so the agent cannot change the state of the environment through the percept.

environment.py is the simulated world. When it is created, it uses the seed to generate the cities with random coordinates and names, computes the distance matrix and randomly chooses the bad roads.

BaselineAgent.py contains the baseline agent.

MemoryAgent.py contains the memory agent.

main.py runs one episode with the seed from **config.json**. First it shows the map, then it runs a simple visit of the cities in index order, the baseline agent and the memory agent, prints every step and finally draws the routes of all three in separate windows.

experiment.py runs the experiments. For seeds 1–20 it creates a new environment and a new agent for each agent and seed, runs one episode and records the total cost, steps, failures, whether the episode was finished and the decision time of the agent. At the end it prints the mean and standard deviation for both agents.

visualization.py contains the drawing functions. **draw_map** draws all cities and all roads, **draw_route** draws the route of one agent with the bad roads, the order of the trips and the base.

config.json contains the parameters of the environment (map size, number of cities, seed, ratio of bad roads, failure probability) and of the experiment (step limit).

requirements.txt lists the external libraries (NumPy and Matplotlib) with their versions, and **README.md** briefly describes the project and how to run it.

3 PEAS description

Performance	total cost of all trips (including failed ones) until all cities are visited and the courier is back at the base; lower is better. Additionally: number of steps, number of failed trips, whether the episode was finished within the step limit.
Environment	map with N cities (random coordinates), every two cities are connected by a road, a part of the roads is bad (hidden), base is city 0.
Actuators	“go to city X ” – one trip per step.
Sensors	current city, coordinates of all cities, distance matrix, set of visited cities, whether the last trip succeeded.

Table 1: PEAS description

4 Environment classification

- **Partially observable** – the agent sees cities and distances, but does not see which roads are bad and what the failure probability is.
- **Stochastic** – the result of a trip on a bad road is random (fails with probability `fail_prob`).
- **Sequential** – every decision changes the position of the courier and the set of visited cities, so it affects all following decisions and the total cost.
- **Static** – the map and the bad roads do not change during an episode, the world does not change while the agent is deciding.
- **Discrete** – finite number of cities, actions and steps.
- **Single-agent** – there is only one courier.

5 Representation of the environment, percepts and actions

5.1 Environment

- `City(idx, x, y, name)` – immutable dataclass, a city is identified by its index in the list of cities,
- distance matrix – Euclidean distances between all cities,
- bad roads – a set of pairs (i, j) ; the number of bad roads is `round(bad_ratio · number of roads)`, chosen randomly by the seed,
- episode state – current city, visited cities, total cost, number of steps, number of failures, result of the last trip.

5.2 Actuator

Method `step(target)`:

1. checks that the city exists and that it is not the current city,
2. adds the distance of the road to the total cost (always, also when the trip fails),
3. if the road is bad, the trip fails with probability `fail_prob` – the courier stays in place,
4. otherwise the courier moves and the city is marked as visited.

5.3 Sensor and percept

Method `get_percept()` returns an immutable `Percept`: `current_city`, `cities`, `visited`, `last_success`, `dist_matrix`. The agent never gets the set of bad roads, the failure probability or the random generator. The agent has no reference to the environment object.

5.4 Episode loop

In every step: environment $\rightarrow$ percept $\rightarrow$ agent $\rightarrow$ action $\rightarrow$ environment. The loop ends when all cities are visited and the courier is at the base, or when `max_steps` is reached.

5.5 Visual representation

For cities, I used 30 cities around Bratislava. Their list you can see in `environment.py` I used `matplotlib` library to visualize them all and routes between them. Here is the image:

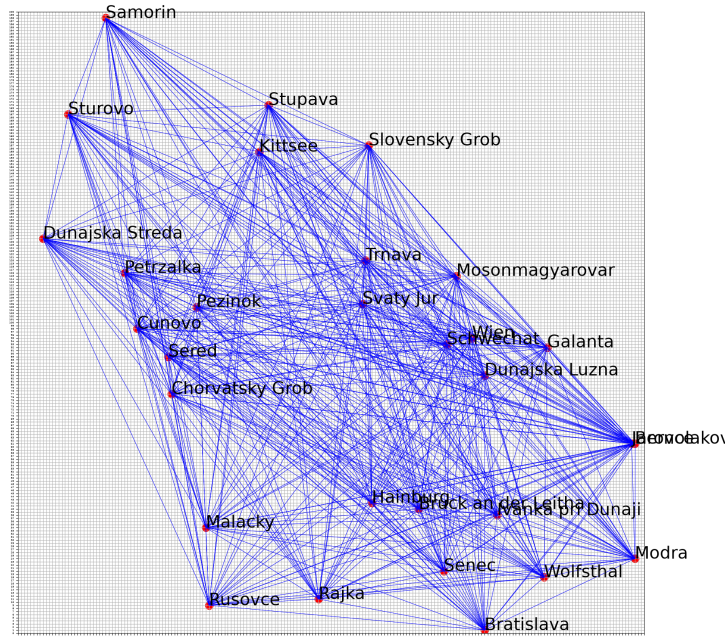


Figure 1: Environment graphic illustration

6 Performance measure

- **total cost** – sum of distances of all trips, including failed ones (main measure),
- **steps** – number of trips,
- **failures** – number of failed trips,
- **completed** – whether the episode ended before `max_steps`,
- **decision time** – time spent in the agent’s `act` method.

7 Baseline agent

Simple reflex agent. In every step it chooses the nearest unvisited city; when all cities are visited it goes to the base. It has no internal state and ignores `last_success`. After a failed trip it is still in the same city, the same city is still the nearest one, so it tries the same road again.

8 Memory agent

8.1 Decision rule

Same rule as the baseline – go to the cheapest unvisited city – but the cost of a road is the *expected* cost based on the agent’s experience:

$$p = \frac{\text{fails}}{\text{attempts} + 1}, \quad \text{cost}(a, b) = \frac{d(a, b)}{1 - p} \quad (1)$$

$\frac{1}{1-p}$ is the expected number of tries needed to get through a road that fails with probability p . A road without experience has $p = 0$ and costs the same as its distance. The +1 in the denominator means that one failure does not make the road “impossible” (and avoids division by zero).

8.2 Example

It’s easier to understand on an example. So here is one: The example comes from `main.py` with the following configuration (`config.json`):

map_Size	cities	seed	bad_ratio	fail_prob	max_steps
200	30	5	0.3	0.7	10000

Table 2: Configuration used in the example

The cities involved are: 0 – Dunajska Streda (base), 10 – Chorvatsky Grob, 24 – Ivanka pri Dunaji, 29 – Modra, 2 – Wolfsthal, 18 – Malacky, 9 – Bratislava, 26 – Senec, 20 – Sered. The distances used in the example are:

$$\begin{aligned} d(2, 18) &= 37.216, & d(2, 9) &= 63.246, & d(2, 14) &= 66.370, \\ d(9, 18) &= 28.018, & d(9, 26) &= 19.313, & d(26, 18) &= 45.343 \end{aligned}$$

The roads 2 – 18 and 18 – 9 are bad. The agent does not know this.

8.2.1 What happened

Both agents start the same way. Steps 1–4 are identical: $0 \rightarrow 10 \rightarrow 24 \rightarrow 29 \rightarrow 2$, all trips succeed and after step 4 both agents are in Wolfsthal (2) with total cost 138.8. From Wolfsthal the nearest unvisited city is Malacky (18), so in step 5 both agents try to go there and both fail.

Step	BaselineAgent			MemoryAgent		
	Trip	Result	Total cost	Trip	Result	Total cost
5	2 → 18	FAIL	176.0	2 → 18	FAIL	176.0
6	2 → 18	FAIL	213.2	2 → 9	OK	239.2
7	2 → 18	OK	250.4	9 → 26	OK	258.5
8	18 → 9	FAIL	278.4	26 → 18	OK	303.9
9	18 → 9	FAIL	306.5	18 → 20	OK	340.2
10	18 → 9	OK	334.5			
11	9 → 26	OK	353.8			
12	26 → 20	OK	406.8			

Table 3: Steps 5–12 of both agents on the same map

After step 12 of the baseline and step 9 of the memory agent both agents have visited the same 9 cities (0, 10, 24, 29, 2, 18, 9, 26, 20) and both stand in Sered (20). The baseline has paid 406.8 and made 4 failed trips, the memory agent has paid 340.2 and made only 1 failed trip. The memory agent is 66.6 cheaper and 3 steps faster at the same point.

8.2.2 Cost of the roads at the moment of each decision

The memory agent chooses the unvisited city with the lowest $\text{cost}(a, b) = \frac{d(a, b)}{1 - p}$,

where $p = \frac{\text{fails}}{\text{attempts} + 1}$. In steps 5 and 6 the agent is in Wolfsthal (2). Only the cheapest candidates are shown.

Step 5. The agent has not failed on any road yet, so $p = 0$ for every road:

$$\text{cost}(2, 18) = \frac{37.216}{1 - 0} = 37.216, \quad \text{cost}(2, 9) = \frac{63.246}{1 - 0} = 63.246, \quad \text{cost}(2, 14) = 66.370$$

37.216 is the lowest, so the agent goes to Malacky (18), the same choice as the baseline. The trip fails.

Step 6. At the beginning of **act** the agent updates its memory with the result of step 5: road (2, 18) has 1 attempt and 1 failure.

$$p(2, 18) = \frac{1}{1 + 1} = 0.5, \quad \text{cost}(2, 18) = \frac{37.216}{1 - 0.5} = 74.432$$

$$\text{cost}(2, 9) = 63.246, \quad \text{cost}(2, 14) = 66.370 \quad (\text{no experience, } p = 0)$$

The road to Malacky is now twice as expensive for the agent (74.432) and Bratislava (9) becomes the cheapest city (63.246). The agent goes to Bratislava and the trip succeeds, because the road 2 – 9 is not bad.

Step	attempts(2, 18)	fails(2, 18)	p	cost(2, 18)	cost(2, 9)
5	0	0	0	37.216	63.246
6	1	1	0.5	74.432	63.246

Table 4: How the cost of the road 2 – 18 changed after one failure

8.2.3 Whole episode

Agent	Total cost	Steps	Failures
BaselineAgent	2256.38	46	16
MemoryAgent	1575.36	36	6

Table 5: Whole episode, seed 5

In this case, MemoryAgent saved us $2256 - 1575 = 681$ units(or km for better understanding) in the whole trip. It’s 30.1 percent cheaper than the BaselineAgent route. Of course it’s not happening on every run and every seed, but the experiment part will be below and here i just found a good example to show how agents works and the example that you can test by yourself using the config.json settings mentioned in the start of the section

8.3 Internal state

The baseline agent has no internal state, it decides only from the current percept. The memory agent keeps the following information between steps:

- **attempts** – for every road the agent has tried, how many times it tried it,
- **fails** – for every road, how many of these tries failed,
- **last_from, last_to** – the last move of the agent.

The percept only says whether the last trip succeeded (`last_success`), not which road it was. That is why the agent remembers its last move: at the beginning of the next step it adds one attempt to this road and, if the trip failed, also one failure. A road is stored as a pair with the smaller city first, so $a \rightarrow b$ and $b \rightarrow a$ are the same road.

From this memory the agent computes the expected cost of a road, so a road that failed before looks more expensive and the agent may choose another city. Roads that were never tried are treated as normal roads.

The memory is empty at the start of every episode. Every seed is a different map with different bad roads, so what the agent learned on one map would be wrong on another.

9 Rationality, autonomy and limitations

What is “the right thing”. Deliver to all cities and return to the base with the lowest total cost. Failed trips are part of the cost, so the right thing is not only a short route, but also not wasting trips on roads that fail.

How the agent chooses an action. It goes to the unvisited city with the lowest expected cost $d(a, b)/(1-p)$, where p is estimated from its own experience. When all cities are visited, it goes to the base.

What the agent knows in advance. Positions of all cities, the distance matrix and the base. It does not know which roads are bad, how many there are and what the failure probability is.

What the agent learns during the run. Only from `last_success` in the percept: on which roads it failed and how often. Nothing else about the hidden state is given to it.

Use of previous experience. Inside one episode every failure makes the road more expensive, so the agent can change its decision. Between episodes nothing is kept, because every seed is a different map.

Autonomy. The agent decides only from its percepts and its own memory. It has no access to the environment object and nobody tells it which roads are bad.

Why a good decision may not lead to success. The agent decides by the *expected* cost, but the result of a trip is random. Example from `main.py` with the configuration from the example above, only `seed = 10`. Both agents are in Wien (23) and the nearest unvisited city is Schwechat (8), $d(23, 8) = 8.246$; the road is bad. Both agents fail twice. After two failures the memory agent has $p = 2/3$ and $\text{cost}(23, 8) = 8.246/(1 - 2/3) = 24.739$, which is more

than $\text{cost}(23, 11) = 20.616$, so it goes to Mosonmagyaróvár (11) and reaches Schwechat later from there ($d(11, 8) = 22.204$).

Step	BaselineAgent			MemoryAgent		
	Trip	Result	Total cost	Trip	Result	Total cost
3	23 $\rightarrow$ 8	FAIL	42.8	23 $\rightarrow$ 8	FAIL	42.8
4	23 $\rightarrow$ 8	FAIL	51.1	23 $\rightarrow$ 8	FAIL	51.1
5	23 $\rightarrow$ 8	OK	59.3	23 $\rightarrow$ 11	OK	71.7
6	8 $\rightarrow$ 11	OK	81.5	11 $\rightarrow$ 8	OK	93.9

Table 6: Seed 10: a reasonable decision with a worse result

After step 6 both agents have visited the same cities, but the memory agent paid 12.4 more. The baseline was lucky – its third try succeeded (probability 0.3). The memory agent was lucky too, the road 23 – 11 is also bad, but it did not know it. Its decision was rational for the information it had, and over the whole episode it was still much cheaper (1681.97 against 2701.92), but in this moment it lost. That is why the agents are compared on 20 seeds and not on a single run.

10 Experiments

10.1 Setup

The experiments are run with `experiment.py`. The parameters of the environment are taken from `config.json`:

- `map_Size` – size of the map,
- `cities` – number of cities,
- `bad_ratio` – part of the roads that are bad,
- `fail_prob` – chance that a trip on a bad road fails,
- `max_steps` – step limit for one episode.

For one configuration both agents are run on seeds 1–20. For every seed a new environment and a new agent are created, so both agents get the same map and the memory agent always starts with empty memory. For every run the total cost, number of steps, number of failures and decision time of the agent are recorded, and the mean and standard deviation over the 20 runs are printed.

10.2 Configurations

Three configurations were tested. They differ only in the failure probability of a bad road (`fail_prob`), all other parameters are the same. Below 0.4 both

agents give almost the same results, because there are only a few failures and the memory has little to learn from, so lower values were not used.

Configuration	map_Size	cities	bad_ratio	fail_prob	max_steps	Seeds
easy	200	30	0.3	0.9	10000	1–20
medium	200	30	0.3	0.7	10000	1–20
hard	200	30	0.3	0.4	10000	1–20

Table 7: Experiment configurations

With 30 cities there are 435 roads, so with `bad_ratio` = 0.3 there are 131 bad roads on every map. I chose exactly that rate, because in Slovakia every third road is considered in bad condition [1]

10.3 Results

Table 8 shows the mean and the standard deviation over 20 runs for each agent and configuration. All episodes were finished, the step limit was never reached.

Configuration	Agent	Cost	Steps	Failures	Time [ms]
easy (0.9)	Baseline	4347.6 $\pm$ 2267.3	110.1 $\pm$ 40.7	80.1 $\pm$ 40.7	0.78
	Memory	2907.0 $\pm$ 1567.3	54.1 $\pm$ 11.7	24.1 $\pm$ 11.7	0.53
medium (0.7)	Baseline	1844.6 $\pm$ 613.7	52.2 $\pm$ 13.7	22.2 $\pm$ 13.7	0.40
	Memory	1767.9 $\pm$ 616.8	42.6 $\pm$ 6.2	12.7 $\pm$ 6.2	0.47
hard (0.4)	Baseline	1294.1 $\pm$ 313.4	36.0 $\pm$ 4.2	6.0 $\pm$ 4.2	0.26
	Memory	1301.5 $\pm$ 239.9	34.4 $\pm$ 2.6	4.4 $\pm$ 2.6	0.38

Table 8: Results, mean $\pm$ standard deviation over 20 runs

Every episode needs exactly 30 successful trips (29 cities and the way back to the base), so the number of steps is always 30 plus the number of failures. That is why the standard deviation of steps and failures is the same.

Table 9 compares the agents directly. “Memory better” is the number of seeds on which the memory agent had a lower total cost than the baseline

Configuration	Cost reduction	Failure reduction	Memory better / worse
easy (0.9)	33.1 %	69.9 %	13 / 7
medium (0.7)	4.2 %	43.1 %	11 / 9
hard (0.4)	−0.6 %	26.1 %	7 / 13

Table 9: Memory agent compared to the baseline

Our agent is mostly effective on big rates, starting from 0.7. And if the rate is below it, we still receive the result which is almost equal to the BaselineAgent. Here is the graph to show how cost reduction, failure reduction and step reduction differs with the same conditions on the experiment and the same 1-20 seeds.

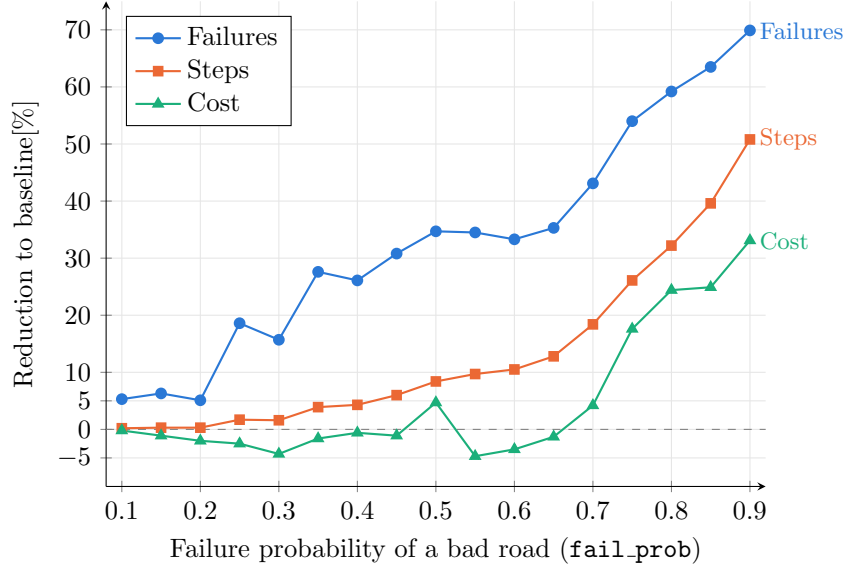


Figure 2: Reduction of failures, steps and cost of the memory agent compared to the baseline

10.4 Interpretation

Why the agents behave differently. Both agents use the same rule, so as long as no trip fails they make exactly the same decisions. With no bad roads they would give identical results. The whole difference comes from what happens after a failure: the baseline stays in the same city, sees the same distances and tries the same road again, while the memory agent sees a more expensive road and often goes somewhere else.

Failures drop more than cost. The memory agent always has fewer failures (26–70 %), but the cost drops much less. A detour is not free – the other city is usually farther than one more try on the short road. Steps drop less than failures, because every episode needs 30 successful trips anyway (steps = 30 + failures).

Effect of fail_prob. With a low failure probability a bad road usually passes after one or two tries, so a retry is cheap and the detour often costs more. The cost of both agents is then almost the same, sometimes the memory agent is

even slightly worse (Figure 2, values below 0.7). With a high probability the baseline can get stuck on one road for many tries – at 0.9 it made 80 failures on average – and avoiding those roads saves a third of the cost.

When the memory agent did not succeed. At `fail_prob` below 0.5 and on (0.5, 0.7) the memory agent was more expensive on most of 20 seeds. And on the 0.5 it's success is probably just luckiness. Failures are rare there, so the main effect of the memory is that one failure doubles the cost of a road and sends the agent to a farther city, while a second try would have been cheaper. Even at 0.9 it was worse on 7 seeds, mostly because of luck: the baseline passed a bad road quickly, or the memory agent's detour went over another bad road it did not know about (the seed 10 example in the previous section). The standard deviation of cost is large in all configurations, because every seed is a different map, so single runs say little – only the mean over 20 seeds shows the trend.

11 Weaknesses and possible improvements

- the first failure on each bad road cannot be avoided, but we can't give out our information about environment, because it will be a strong rule violation
- the greedy rule is not optimal – a planning algorithm (e.g. 2-opt) could be added,
- the estimate of p is simple – a better estimate could be used.

12 Ending and main point of project

In this project I was trying to implement an agent, that will give us better results in the same conditions compared to basic rule: "choose cheapest". I have done it without changing the main structure of each agent and without giving someone internal environment information (about bad roads, for example). Real world apply for that project might be narrow, but in my opinion, it's really useful, because it helps to understand how basic navigation systems work. For example, if we change our percepts to users on some app, and majority of couriers will fail on a bad road, for future, our app will consider this route as bad and will assign the proper cost for moving on it.

At the end I wanted to mention that I used AI to make some analysis and for graphs and tables in this project. And I also uploaded everything on the github repo, where you can see the progress of the project through stages. It will be in references below [4]

13 Libraries and external sources

- Python 3.14
- NumPy – distance matrix
- Matplotlib – visualization
- standard library: `random`, `math`, `json`, `statistics`, `time`, `dataclasses`

References

- [1] Standart.sk date of advising is 04.10.2026 <https://standard.sk/1324082/statne-cesty-zaostavaju-za-zupnymi-v-siedmich-z-osmich-krajov-su-v-horsom-stave>
- [2] NumPy documentation. <https://numpy.org/doc/>
- [3] Matplotlib documentation. <https://matplotlib.org/stable/>
- [4] Github repository https://github.com/mohula-m/simple_agent